CHANGWON NATIONAL UNIVERSITY

Report of Summer Term Internship 2026

Better Action Heads for Robot Vision-Language-Action Models: An Operator-Learning and Topology View

Ayush Shah

Lab: ISLAB · Supervisor: Dr. Oh Kwon Seol

Table of Contents

Abstract

This project is about teaching robots to move better. Modern robot “brains” are large vision-language-action (VLA) models: you show them camera images and tell them what to do in plain language, and they output a short sequence of actions for the arm. The part that actually produces the numbers for the motion is called the *action head*. Most strong models use a heavy action head based on diffusion or flow matching, which builds the motion by slowly cleaning up random noise over many steps.

Coming from a mathematics and computing background, we noticed that this problem has a very clean mathematical shape. A short piece of robot motion is really a *function of time*, and predicting a function from an input is exactly what a branch of maths called *operator learning* studies. There is a neural network built for this, the **DeepONet**, which comes straight out of a mathematics paper. We also noticed that a motion has a *shape* — it lives in space and has geometric structure — and there is a field of maths, *topology* (specifically *persistent homology*), whose whole job is to describe shape. So we asked two questions: (1) if we replace the heavy diffusion/flow head with a light DeepONet operator head, do we keep the accuracy while getting a smaller and faster model? and (2) if we add a topology-based loss that asks the predicted motion to have the same *shape* as the expert’s, does the robot become more reliable when the world changes?

We built these as drop-in action heads and compared them fairly against the original heads on the LIBERO benchmark and its harder distribution-shift version, LIBERO-Plus. We ran two full studies — first on **SmolVLA**, then on **ACT** — and their measured numbers and plots are reported in place, right after each study. Two much larger models (**pi0.5** and **GR00T N1.6**) were added only to compare against the strongest available systems and for completeness; those runs are still going, so their tables are left blank.

In short: the operator head is much smaller and faster; on ACT it is better on every average measure, and on SmolVLA its clear win is robustness and efficiency while in-distribution accuracy is a statistical tie. The persistent-homology loss, though mathematically attractive, did not help, so we do not recommend it. We report all of this honestly, including where our method does not win.

1. Introduction and motivation

1.1 Where the idea came from

We are students in a Mathematics and Computing department, so when we started reading about robot learning we did not look at it the way a pure robotics person might. We kept seeing two things that felt like they belonged to us.

The first was the action itself. A robot policy does not usually output one motor command. It outputs a short *chunk* — for example the next 50 little steps of the arm. If you write the chunk as $a(\tau)$ with τ running from 0 (start) to 1 (end), then the model's real job is to take an observation and return a whole *function* $a(\tau)$. Producing a function from an input is not ordinary regression (vector to vector); it is an *operator* (a map from inputs to functions). There is a well-known result in mathematics, the universal approximation theorem for operators (Chen and Chen, 1995), and a neural network built directly on it, the DeepONet (Lu et al., 2021). That is a maths paper being used for a robotics problem, which is exactly the kind of bridge we wanted to build.

The second was that a motion has a *shape*. If you plot the sequence of actions as a small cloud of points, an expert demonstration and a clumsy imitation might pass through different exact points but still have very different overall geometry — one smooth and compact, the other jittery and spread out. The standard training loss (mean squared error, MSE) only checks each time step on its own; it never looks at the global shape. Topology, and in particular *persistent homology* (Edelsbrunner, Letscher and Zomorodian, 2002; Carlsson, 2009), is the mathematics of shape. So we asked whether adding a topology-aware term could make the motion globally more sensible.

So the whole project is really: take two clean mathematical ideas — **operator learning** and **topology** — and see if they help a very practical thing, a robot arm doing tasks it was trained on and tasks where the world has been changed a little.

1.2 What we actually did

We took the action head out of several VLA models and replaced it with our own operator head (DeepONet), and separately added a persistent-homology loss, always keeping everything else the same so that any difference comes from the head alone. We trained and evaluated on LIBERO (four task suites) and on LIBERO-Plus (seven kinds of world change). We ran the two full studies on **SmolVLA** (~450M) and **ACT** (~88M), then ported the same operator head to **pi0.5** (~3.3B) and **GR00T N1.6** (~3B) for a state-of-the-art comparison and completeness. We did not tune the test harness to get a nice answer, and we report whatever the numbers turn out to be.

2. Background

2.1 What is a vision-language-action model?

A VLA model is a single neural network that takes what the robot sees (camera images), what it is told to do (a sentence), and where it currently is (arm and gripper state), and outputs the next actions. Figure 1 shows the general picture that all our models follow.

Figure 1. A vision-language-action (VLA) model, at a glance

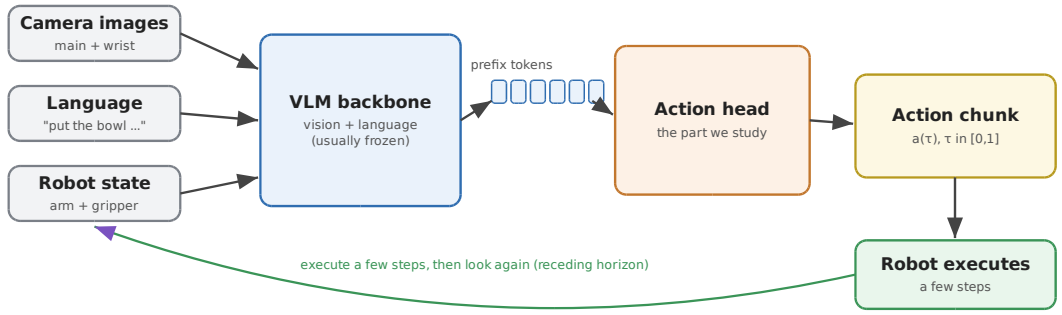


Figure 1. A vision-language-action model, at a glance. Images, language and robot state go into a large vision-language backbone, which turns them into "prefix tokens". The action head reads those tokens and produces a short chunk of motion, which the robot executes a few steps of before looking again (receding-horizon control).

The key split for us: the **backbone** is the big vision-and-language part (taken from a pretrained model), and the **action head** is the small part that turns the backbone's understanding into actual motion numbers. The action head is the part we study and replace. Everything except the head is kept

identical between the variants we compare, so any difference is due to the head.

2.2 The benchmark: LIBERO and LIBERO-Plus

We test on **LIBERO**, a standard simulated robot-arm benchmark with four *suites* of ten tasks each: **Spatial** (depends on *where* things are), **Object** (depends on *which* object), **Goal** (explicit goal), and **Long** (long-horizon tasks that chain several steps; the hardest). For robustness we use **LIBERO-Plus**, which perturbs the same tasks in seven ways (Figure 12). A model is trained on the normal world and tested on each perturbed version it has never seen; the robustness score is the average success over the seven categories.

Figure 12. LIBERO-Plus: seven ways the world is changed to test robustness

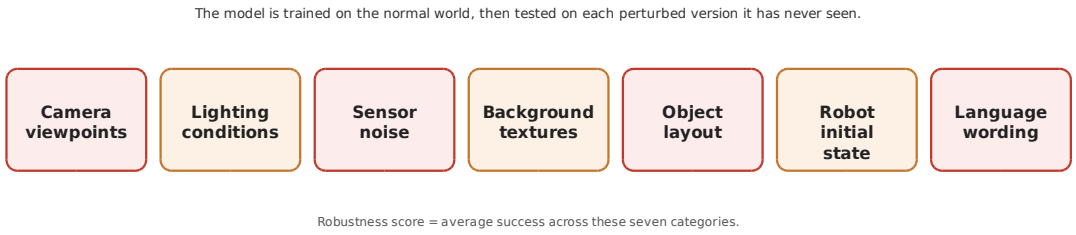


Figure 12. LIBERO-Plus changes the world in seven ways to test out-of-distribution robustness: camera viewpoint, lighting, sensor noise, background textures, object layout, robot initial state, and the wording of the instruction.

Throughout, we measure **in-distribution accuracy**, **robustness** (LIBERO-Plus, averaged over the seven categories and also broken down per category), the **action-head parameter count**, and the **inference latency**.

3. The two mathematical ideas

3.1 Persistent homology (topology of a trajectory)

Persistent homology comes from topological data analysis. It describes the *shape* of a set of points: how many separate clusters, whether there are loops or holes, and how “big” these features are. Imagine growing little balls around each point; as they grow, points connect, clusters merge, loops appear and disappear. Features that survive over a wide range of ball sizes are “persistent” (real); short-lived ones are noise.

For a robot, think of one predicted action chunk as a cloud of points (one per time step); the expert’s chunk is another cloud. MSE compares them

point by point, but persistent homology compares their *overall shape* (Figure 4).

Figure 4. The persistent-homology (PH) idea: compare the SHAPE of trajectories

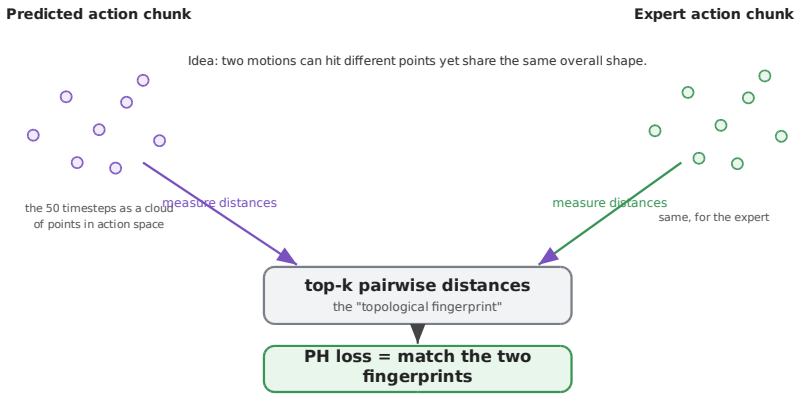


Figure 4. The persistent-homology idea. We treat the predicted and expert action chunks each as a small cloud of points, measure the pattern of pairwise distances (a "topological fingerprint"), and ask the two fingerprints to match. This looks at the global shape of the motion, not each time step on its own.

Computing true persistent homology every training step is slow and its gradient is awkward, so we use a cheap, differentiable *surrogate*: take the sorted top-k largest pairwise distances of the predicted and expert chunks (the "fingerprint") and penalise their difference. It is quick, differentiable, and captures multi-scale spread. We compute it only over the real action dimensions, and only during training (Figure 5) — at test time it does nothing, so it never slows the robot down.

Figure 5. PH is an extra training-time loss (inference is untouched)

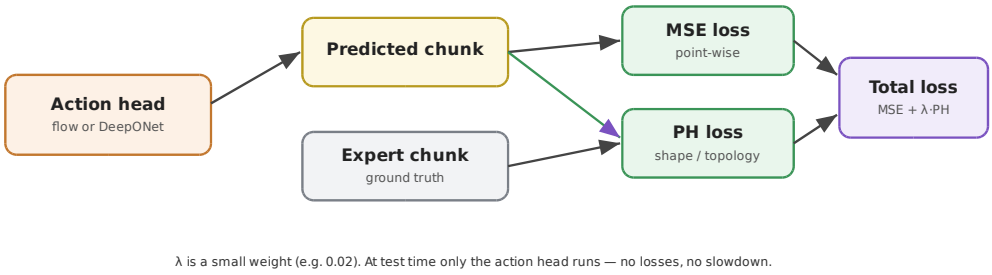


*Figure 5. The persistent-homology loss is an extra training-time term: total = MSE + lambda*PH. Inference uses only the action head, so there is no extra cost at test time.*

3.2 DeepONet (operator learning), and why cross-attention

An ordinary network maps a vector to a vector. A DeepONet maps an input to a whole *function*, using two sub-networks: a **branch** that reads the observation and outputs coefficients c, and a **trunk** that reads a query time $\tau \in [0,1]$ and outputs basis values $\varphi(\tau)$. The predicted motion is

$$a(\tau) \approx \mathrm{OutMLP}\big(c \odot \varphi(\tau)\big),$$

where $\odot$ is element-wise multiplication. The branch decides *what to do*; the trunk gives time-shapes; multiplying combines the "what" with the "when". This is backed by a universal approximation theorem for operators, and the whole chunk comes out in **one forward pass** — no denoising loop — so the head is small (~10-11M parameters) and fast.

Why the cross-attention pooler. For the branch to decide *what to do*, it first has to *see* the observation well. The backbone hands us a long sequence of tokens — one for each image patch and each word. A simple way to summarise them is to average them into a single vector, but averaging blurs everything together and loses *where* things are, which is exactly what a spatial task needs (is the bowl on the stove or in the cabinet?). Instead we use a small **cross-attention pooler**: a handful of learned "query" vectors look across *all* of the tokens and each pulls out the piece it cares about (Perceiver-style). This lets the DeepONet head look at the whole input properly and keep the spatial detail — it can attend to the exact object and location that matter — before the branch turns that into coefficients. Figure 6 shows the full head.

Figure 6. Inside the DeepONet operator action head

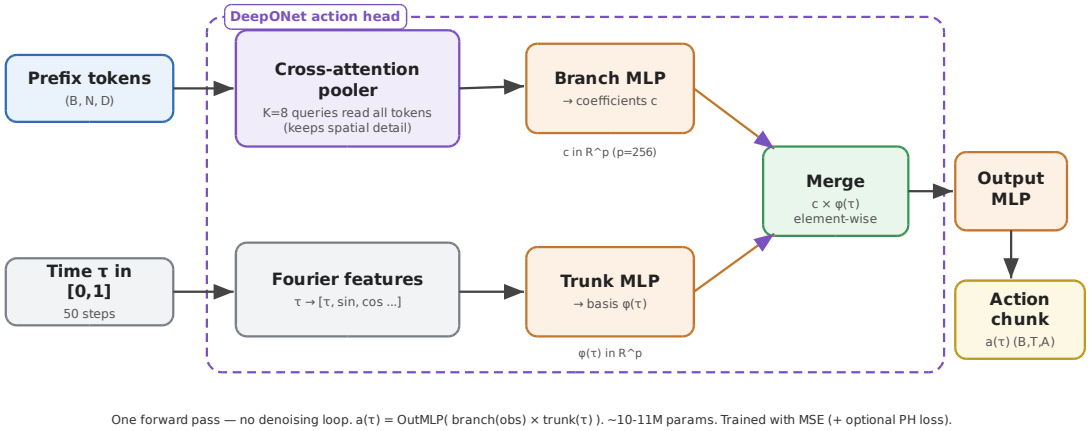


Figure 6. Inside the DeepONet operator action head. The branch (top) reads the prefix tokens through a cross-attention pooler (which lets the head see the whole input and keep spatial detail) and a small MLP that outputs

coefficients c. The trunk (bottom) turns the query time into Fourier features and outputs basis values. Their element-wise product goes through an output MLP to give the whole action chunk in one pass.

4. How every model is trained and tested

Every model follows the same recipe (Figure 11): a pre-training stage on all 40 tasks (four suites together), then a per-suite fine-tuning stage on each suite's 10 tasks, then evaluation twice — once in-distribution and once on LIBERO-Plus. For the large models (SmolVLA, pi0.5, GR00T) the backbone is frozen and only the head trains; ACT is small enough that the whole model is trained. In every case the **only thing that differs between the three variants of a model is the action head**, so the comparison is fair. We use **receding-horizon control** (predict a chunk, execute a few steps, re-observe) with the same replan interval for everyone, and an exponential moving average of weights for evaluation.

Figure 11. The training and evaluation protocol (same for every model)

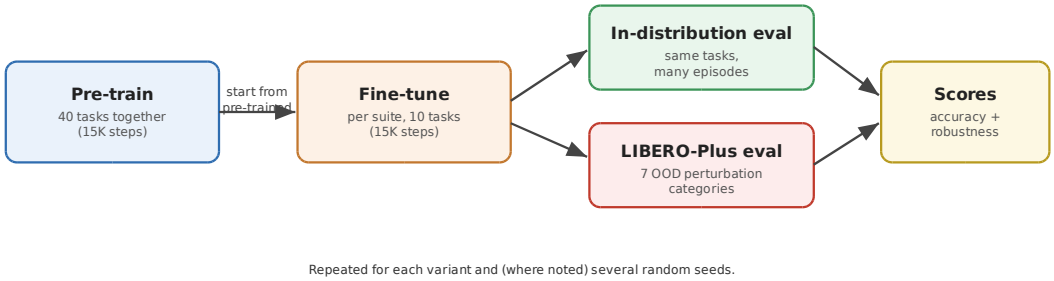


Figure 11. The training and evaluation protocol, identical for every model: a 40-task pre-training stage, a per-suite fine-tuning stage, then evaluation on the normal tasks and on the seven LIBERO-Plus perturbation categories.

Throughout the results, "M1 = flow, M3 = DeepONet, M4 = DeepONet + PH". All runs are on a single RTX PRO 6000 (Blackwell) GPU in bfloat16. Higher is better everywhere; numbers are success rates in percent.

5. Study 1 — SmolVLA

5.1 The SmolVLA model and the three heads

SmolVLA has a frozen vision-language backbone (SmolVLM2, ~350M) and, by default, a flow-matching action head (~100M). Figure 2 shows the model with our DeepONet head in place (the boundary marks the head we swap; the outer boundary marks the whole model).

Figure 2. SmolVLA with the DeepONet action head

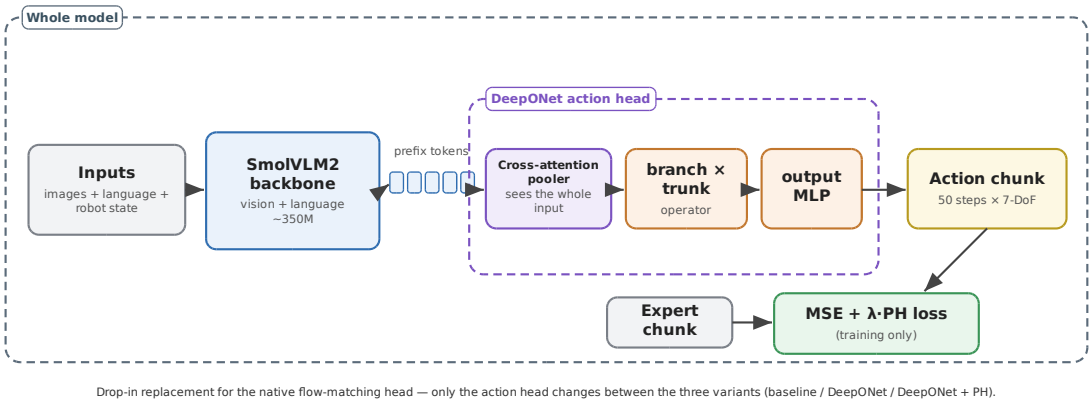


Figure 2. SmolVLA with the DeepONet action head. The inner dashed boundary is the part we replace; the outer boundary is the whole model. The persistent-homology loss is shown as a training-only term.

The baseline head is flow matching (Figure 3): given the prefix tokens, a noisy action and a time value, it predicts a *velocity* that points from noise towards the real action, and the clean action is recovered by one formula. We call this baseline **M1**. We then compare it against the DeepONet operator head (**M3**) and the DeepONet head with the added persistent-homology loss (**M4**, Figure 7).

Figure 3. How the flow-matching action head works

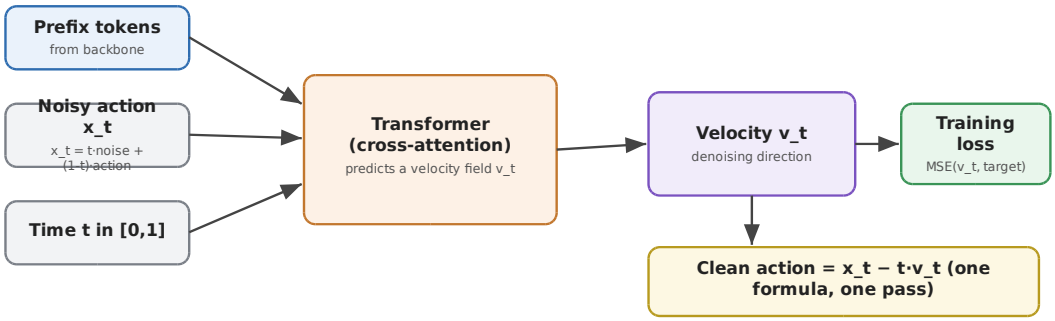


Figure 3. The flow-matching baseline head. It predicts a velocity field; because the noise-to-action path is a straight line, the clean action is recovered by a single formula.

Figure 7. DeepONet + PH variant (same head, extra training loss)

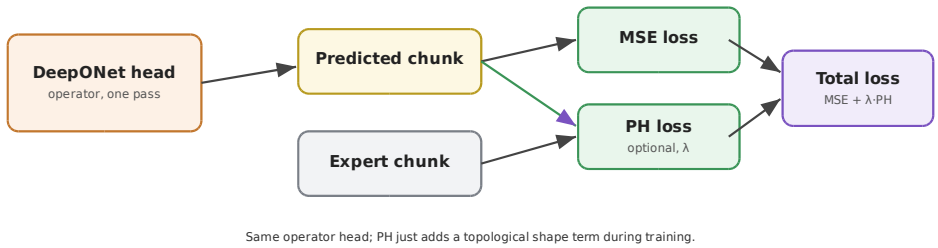


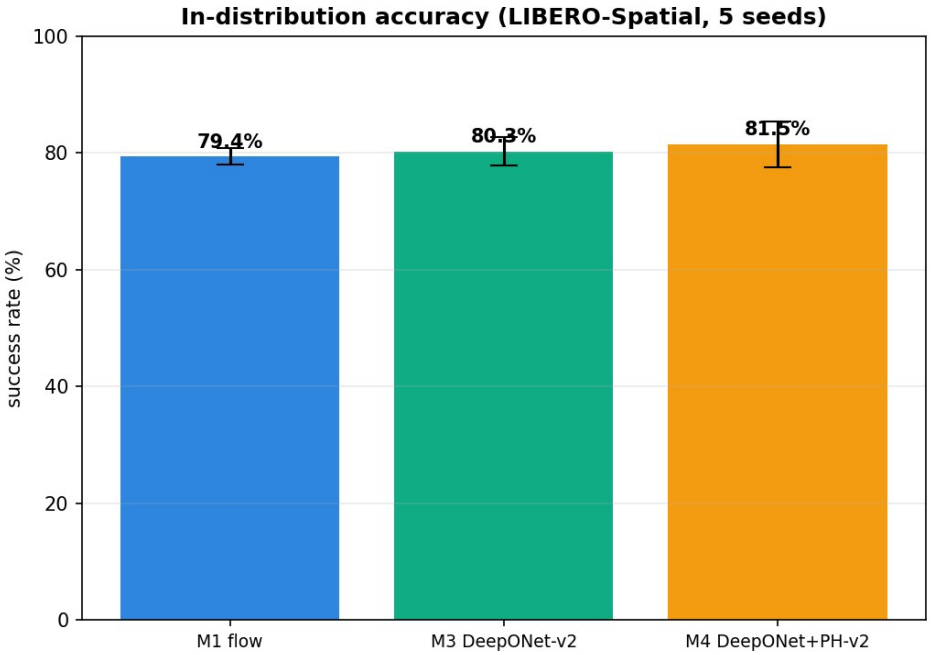
Figure 7. The DeepONet + PH variant (M4). The head is the same operator head; during training we add the persistent-homology shape loss on top of the usual MSE.

5.2 SmolVLA results

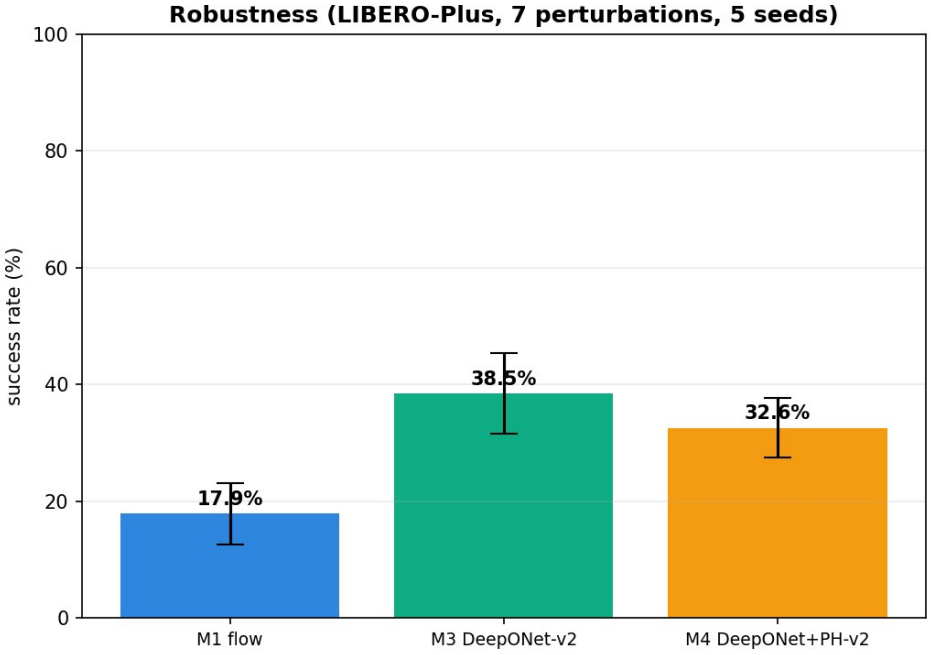
5.2.1 Headline — LIBERO-Spatial, 5 seeds

Model	In-dist (%)	Robustness (%)	Latency (ms)	Head params (M)
M1 flow (baseline)	79.4 ± 1.4	17.9 ± 5.2	148.1	99.9
M3 DeepONet	80.3 ± 2.4	38.5 ± 6.9	29.5	10.4
M4 DeepONet + PH	81.5 ± 3.9	32.6 ± 5.1	29.5	10.4

The operator head is **9.6× smaller** and **~5× faster** than the flow head. In-distribution accuracy is essentially the same as the baseline; the clear difference is robustness, where DeepONet is **more than 2× the baseline** (38.5 vs 17.9).



In-distribution accuracy on LIBERO-Spatial (5 seeds): the three heads are within noise of each other.



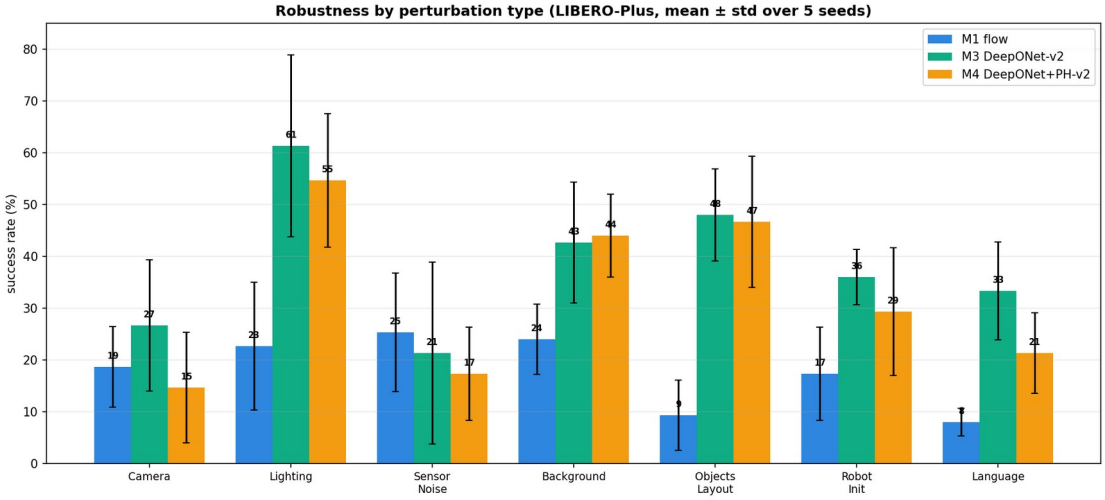
Robustness on LIBERO-Plus (5 seeds): DeepONet (M3) is more than twice the flow baseline; adding PH (M4) lowers it again.

5.2.2 Robustness by perturbation (Spatial)

Perturbation	M1 flow	M3 DeepONet	M4 DeepONet +

			PH
Camera Viewpoints	18.7	26.7	14.7
Light Conditions	22.7	61.3	54.7
Sensor Noise	25.3	21.3	17.3
Background Textures	24.0	42.7	44.0
Objects Layout	9.3	48.0	46.7
Robot Initial States	17.3	36.0	29.3
Language Instructions	8.0	33.3	21.3
Average	**17.9**	**38.5**	**32.6**

DeepONet wins 6 of the 7 categories (flow only wins Sensor Noise); the biggest gains are Objects Layout (+38.7) and Language (+25.3).



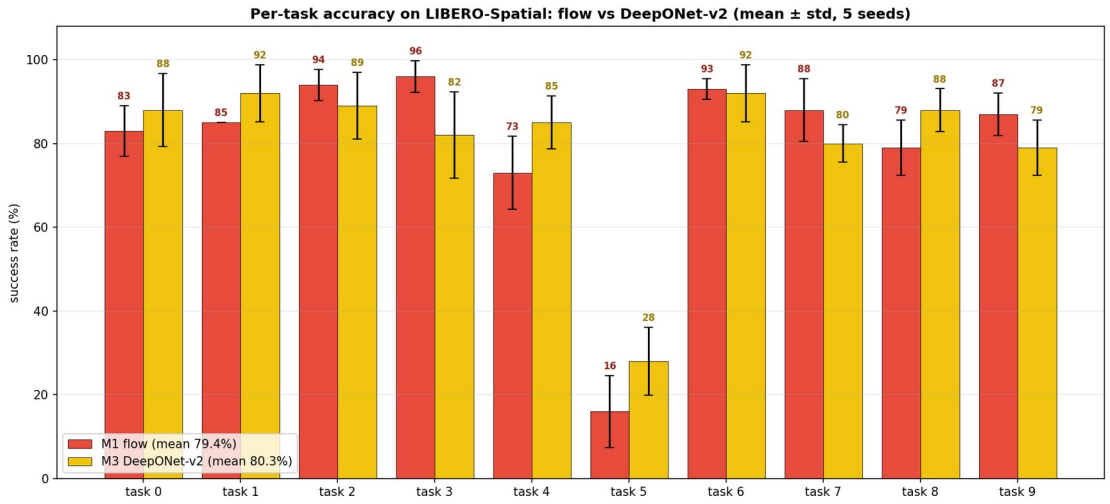
Robustness by perturbation type (LIBERO-Spatial, 5 seeds). DeepONet leads on six of seven perturbations.

5.2.3 Per-task in-distribution (Spatial, 5 seeds)

Task	M1 flow	M3 DeepONet	M4 DeepONet + PH
task0	83.0	88.0	95.0
task1	85.0	92.0	92.0
task2	94.0	89.0	90.0
task3	96.0	82.0	84.0
task4	73.0	85.0	88.0

task5 (hard “stacked-bowl”)	16.0	28.0	38.0
task6	93.0	92.0	90.0
task7	88.0	80.0	83.0
task8	79.0	88.0	84.0
task9	87.0	79.0	71.0

Even on the hard task5, the operator heads (28/38%) beat flow (16%).



Per-task in-distribution success on LIBERO-Spatial: DeepONet-v2 vs flow, task by task.

5.2.4 Ablations, and the module we settled on

We took the operator head apart to see which piece matters (LIBERO-Spatial):

Configuration	In-dist (%)	Robustness (%)	Seeds
Full DeepONet-v2 (p=256, 3 cross-attn blocks, Fourier)	80.3 ± 2.4	38.5 ± 6.9	5
basis size p 256 -> 64	81.7 ± 0.5	32.7 ± 5.2	3
linear time features (no Fourier)	82.7 ± 1.4	39.0 ± 8.7	3

cross-attention blocks 3 -> 1	78.2 ± 2.0	30.8 ± 4.5	3
plain regression head (no operator merge)	83.0 ± 4.2	32.7 ± 0.9	3

Reading this honestly: for raw in-distribution accuracy the pieces barely matter (a plain regression head on the same context even scores highest). What clearly moves is **robustness**, and the two components that carry it are the **cross-attention pooler** (three blocks — the head that sees the input best is the most robust) and the **operator merge** (the branch-times-trunk structure, robustness 38.5 vs the regression head's 32.7). Based on this, the **final module we use everywhere is: DeepONet head with the cross-attention pooler (3 blocks), basis size p = 256, Fourier time features, and no PH loss.** We keep Fourier features because, although they barely change these numbers, they help on the harder long-horizon suite where the motion has more temporal structure.

5.2.5 Per-suite in-distribution (single seed, 15K steps)

Suite	M1 flow	M3 DeepONet	M4 DeepONet + PH
LIBERO-Spatial	78.5	82.0	80.5
LIBERO-Object	84.5	94.0	87.0
LIBERO-Goal	93.5	90.0	89.0

5.2.6 Fixed-budget vs best-checkpoint (all four suites, 30K)

Selection	Model	Spatial	Object	Long	Goal	4-suite avg
fixed 30K	M1 flow	79.5	87.5	66.5	93.5	81.75
fixed 30K	M3 DeepONet	85.0	87.0	58.5	90.0	80.12
fixed 30K	M4 DeepONet + PH	82.5	89.5	60.5	89.0	80.38
best per suite	M1 flow	79.5	84.5	66.5	93.5	81.00
best per suite	M3 DeepONet	85.0	94.0	58.5	90.0	81.88
best per suite	M4 DeepON	82.5	87.0	60.5	89.0	79.75

	et + PH					

A caveat we want to be upfront about: at a **fixed** 30K budget the flow baseline slightly wins the four-suite average (81.75 vs 80.12); DeepONet only edges ahead (81.88) if you pick the best checkpoint per suite, which is selection-dependent. So across suites the in-distribution story is “roughly a tie”.

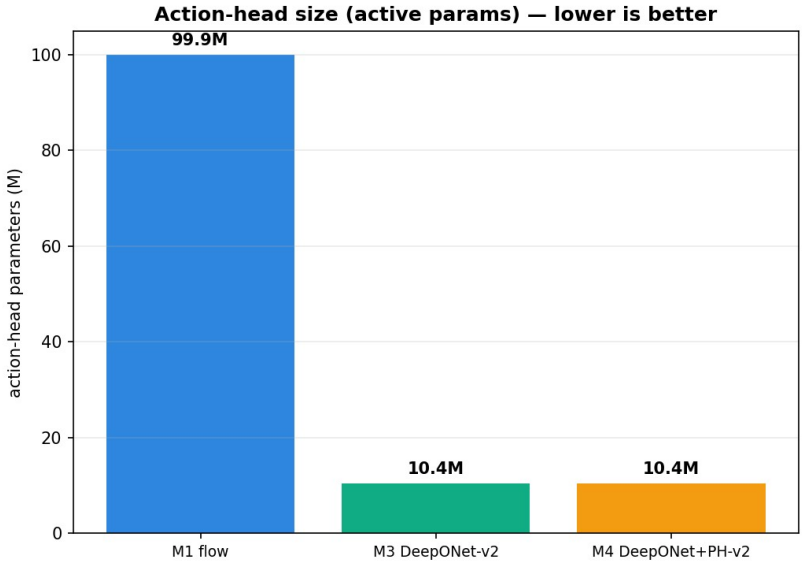
5.2.7 Statistical significance (paired tests)

Comparison	Metric	Mean diff (pp)	p-value	Significant?
DeepONet vs flow	in-dist	+0.9	0.61	no (a tie)
DeepONet vs flow	robustness	+20.6	1.1e-06	yes
DeepONet + PH vs flow	robustness	+14.7	1.3e-04	yes
DeepONet + PH vs DeepONet	robustness	-5.9	0.032	yes (PH hurts)
DeepONet-v2 vs DeepONet-v1	in-dist	+28.5	5.8e-08	yes

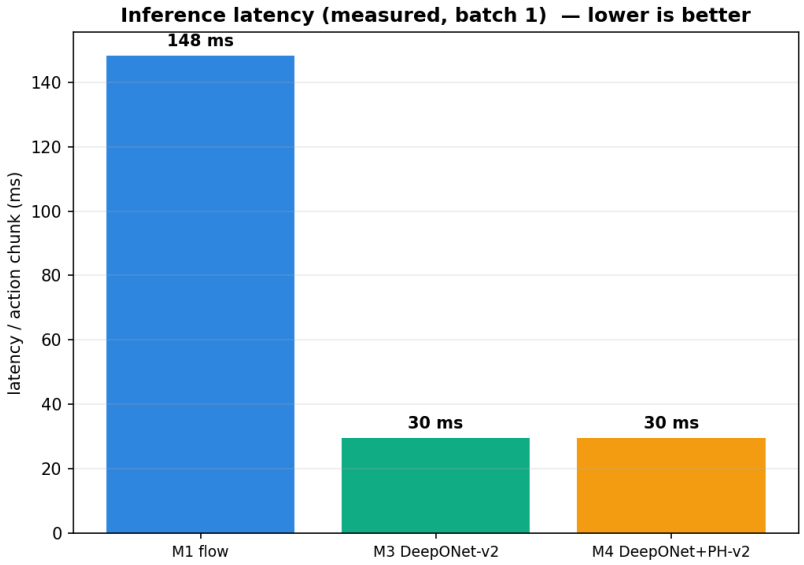
Plainly: the in-distribution DeepONet-vs-flow difference is **not** significant (a tie), but the robustness difference **is** (a real, large win). Adding PH **significantly reduces** robustness, which is why our final module leaves it off. The last row is our own earlier head (version 1), which was much weaker in-distribution; version 2, with the cross-attention pooler, fixed that.

5.2.8 Efficiency, and a generalisation check

The operator head is far cheaper — 10.4M vs 99.9M parameters and about 29.5 ms vs 148 ms per prediction.



Action-head parameter counts for the three SmolVLA heads.



Inference latency: the operator head is roughly five times faster than the flow head.

As a small anti-memorisation check, we evaluated on LIBERO-Goal with the object layout shifted by amounts never seen (offsets 0, 20, 30): success stayed at 91.5%, 88.0% and 91.5%, so the model generalises to layout changes rather than memorising positions.

6. Study 2 — ACT

6.1 Why we moved from SmolVLA to ACT

The SmolVLA study gave a strong robustness result, but it had one weakness we wanted to remove: the robustness numbers were measured on a **single suite** (Spatial), because each SmolVLA run is large and slow. To make the robustness claim trustworthy we needed a model small enough to run the **full four suites with LIBERO-Plus and several seeds**, and ACT is exactly that — a compact, widely-used policy that trains quickly. ACT also comes with its own strong, purpose-built action head (a transformer decoder), so swapping in the DeepONet head is a fair "operator vs a real specialised head" test on a **different architecture**. In other words, moving to ACT lets us check that the operator's benefits are not specific to SmolVLA and that they hold up under a more complete robustness evaluation.

6.2 The ACT model with the DeepONet head

ACT uses a ResNet image encoder and a transformer encoder; its baseline head is a 7-layer transformer decoder (~38M). We swap that for the DeepONet head (~10M) and also try the PH loss (Figure 8).

Figure 8. ACT with the DeepONet action head

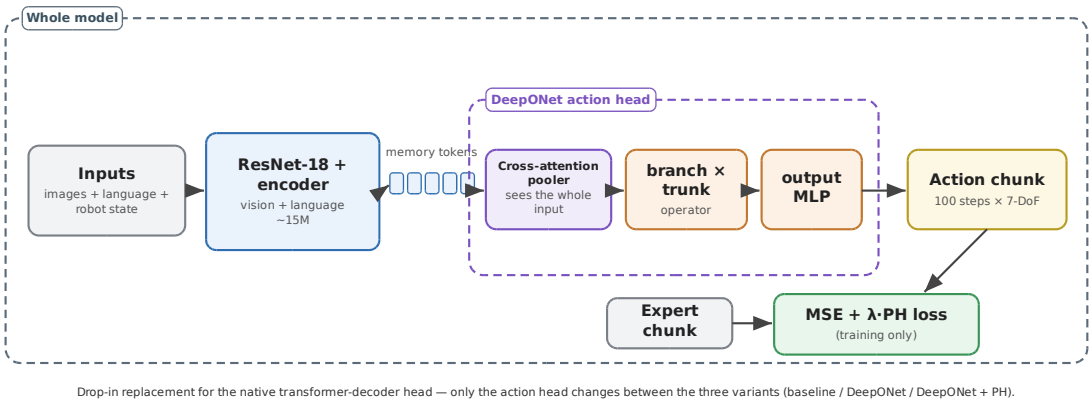


Figure 8. ACT with the DeepONet action head. The inner boundary is the head we replace (the baseline is a transformer decoder); the outer boundary is the whole model; the persistent-homology loss is a training-only term.

6.3 ACT results

Here we evaluated all four suites with three seeds, and we have LIBERO-Plus robustness on all four suites (not just Spatial).

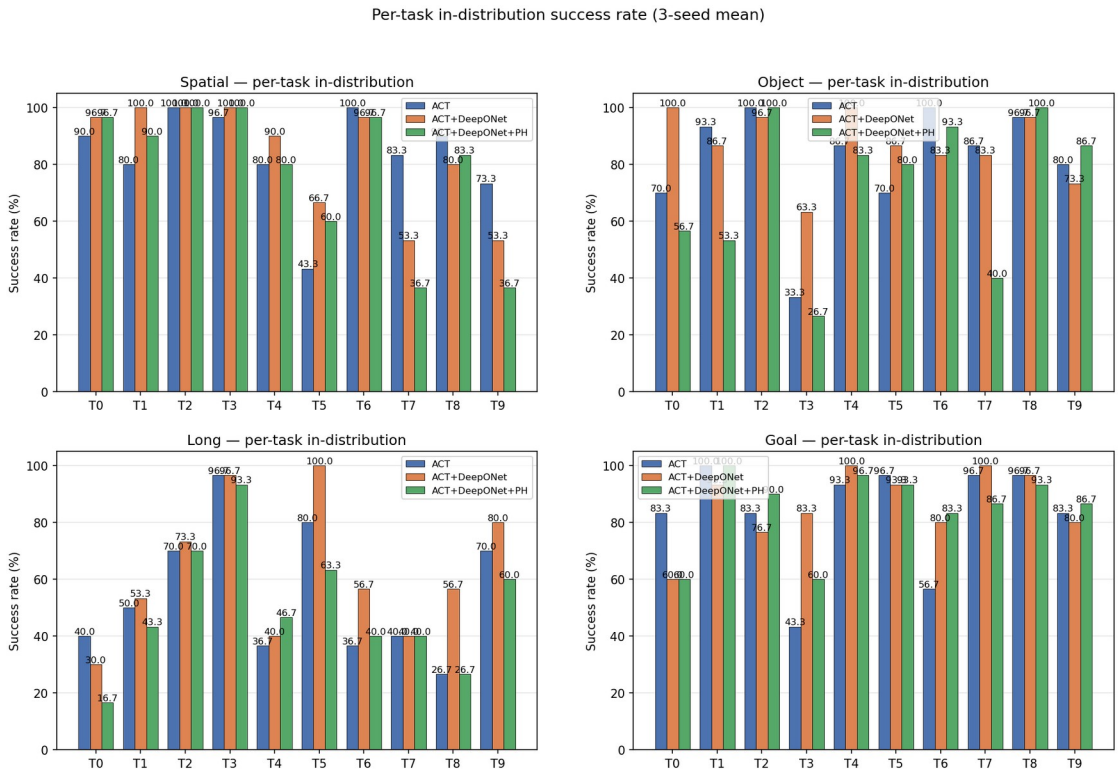
6.3.1 In-distribution (%)

Suite	ACT (baseline)	ACT +	ACT +

		DeepONet	DeepONet + PH
LIBERO-Spatial	83.7	83.7	78.0
LIBERO-Object	81.7	87.0	72.0
LIBERO-Long	54.7	62.7	50.0
LIBERO-Goal	83.3	86.3	85.0
Average	**75.9**	**79.9**	**71.3**

DeepONet is at least as good as the baseline on every suite (a tie on Spatial, wins on the other three), at a 3.7× smaller head. PH lowers accuracy on every suite.

In-distribution success per suite on ACT, with 3-seed error bars.

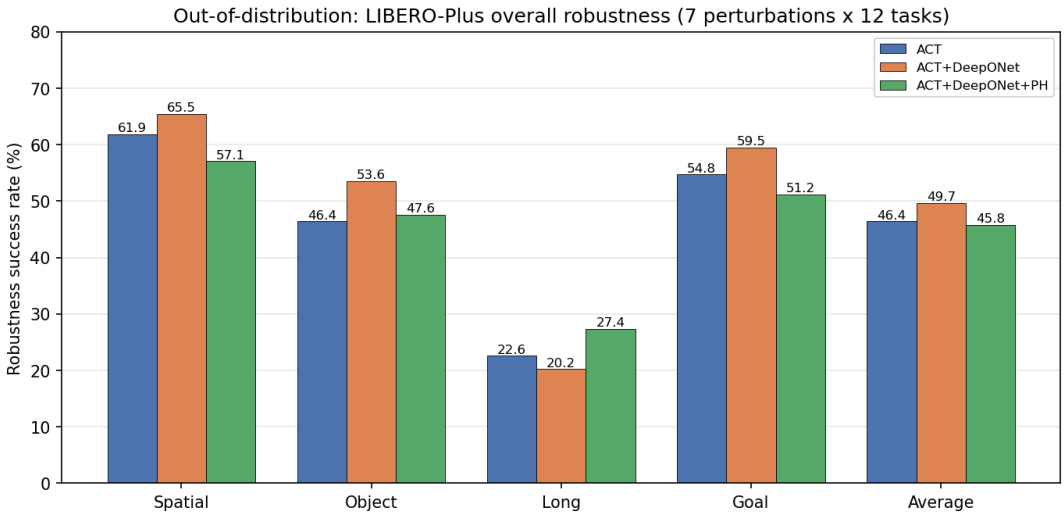


Per-task in-distribution success, all four suites (ACT).

6.3.2 Robustness — LIBERO-Plus (%)

Suite	ACT (baseline)	ACT + DeepONet	ACT + DeepONet + PH
LIBERO-Spatial	61.9	65.5	57.1
LIBERO-Object	46.4	53.6	47.6
LIBERO-Long	22.6	20.2	27.4
LIBERO-Goal	54.8	59.5	51.2
Average	**46.4**	**49.7**	**45.8**

DeepONet is the most robust on average (49.7 vs 46.4), winning three of four suites. PH’s only win is Long.

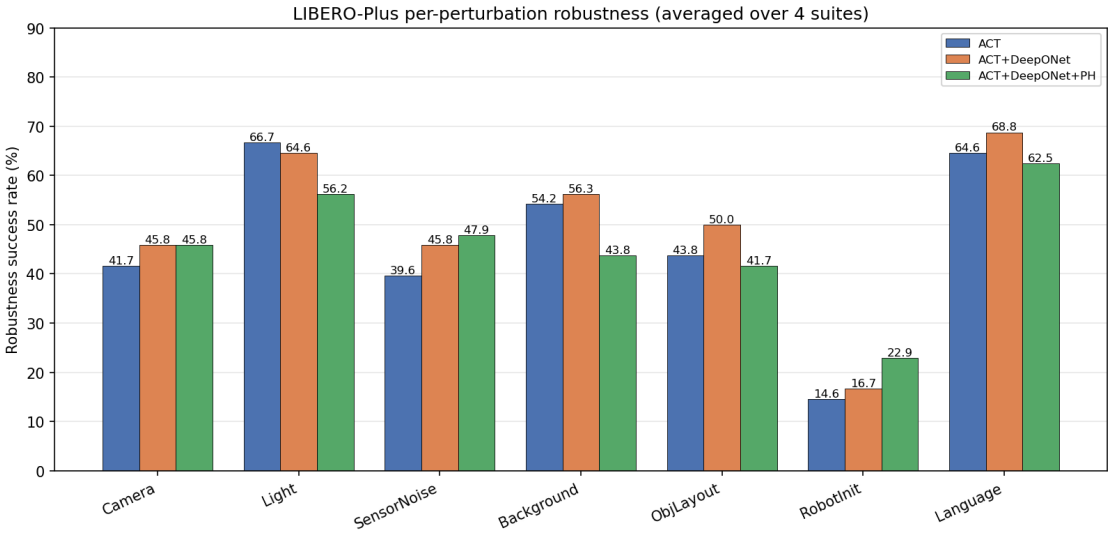


LIBERO-Plus robustness per suite (ACT).

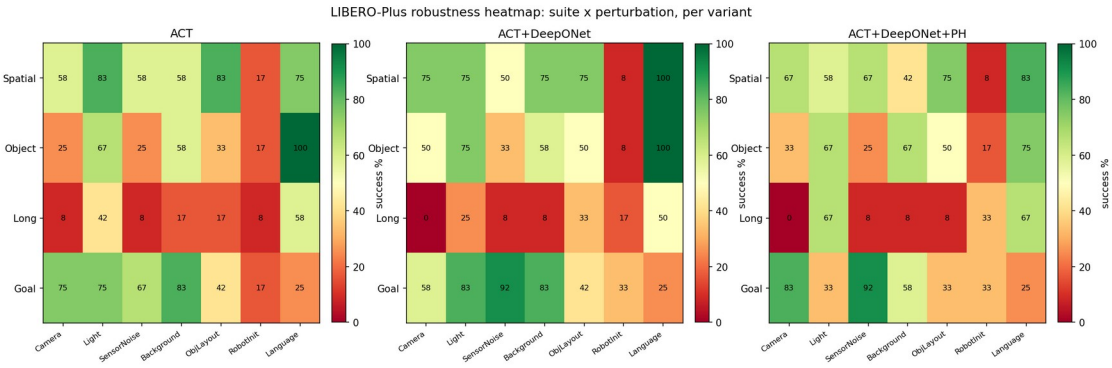
6.3.3 Robustness by perturbation (averaged over the four suites)

Perturbation	ACT	ACT + DeepONet	ACT + DeepONet + PH
Camera Viewpoints	41.7	45.8	45.8
Light Conditions	66.7	64.6	56.2
Sensor Noise	39.6	45.8	47.9
Background Textures	54.2	56.3	43.8
Objects Layout	43.8	50.0	41.7
Robot Initial States	14.6	16.7	22.9
Language Instructions	64.6	68.8	62.5

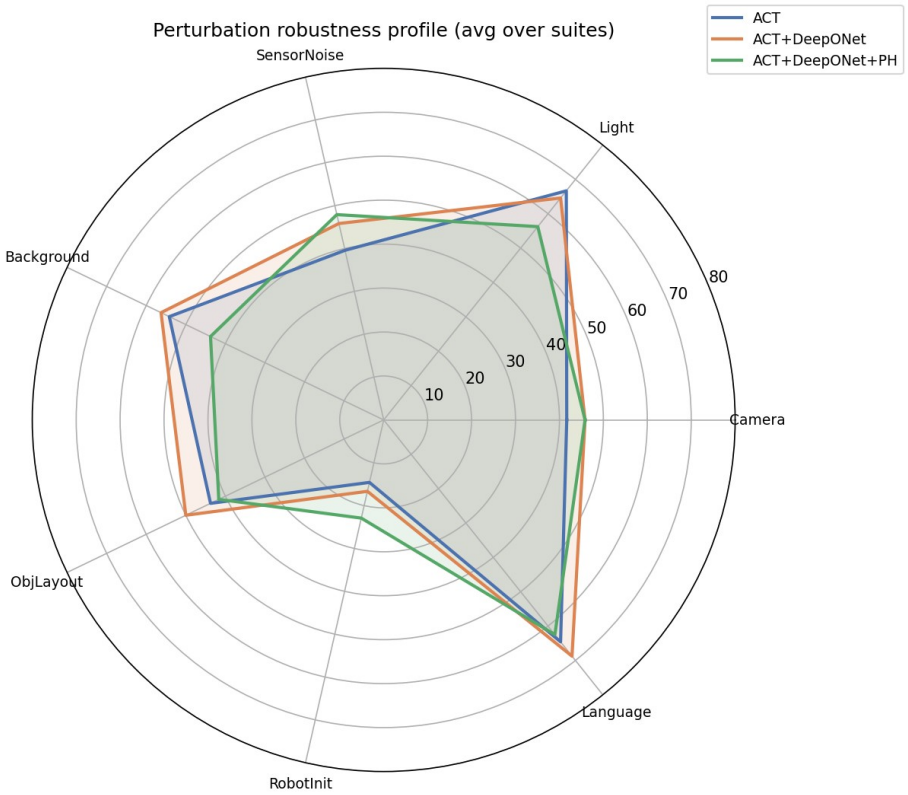
DeepONet wins four of seven categories. “Robot Initial States” is the universal weak spot for everyone (~15-23%).



Robustness by perturbation category, averaged over suites (ACT).



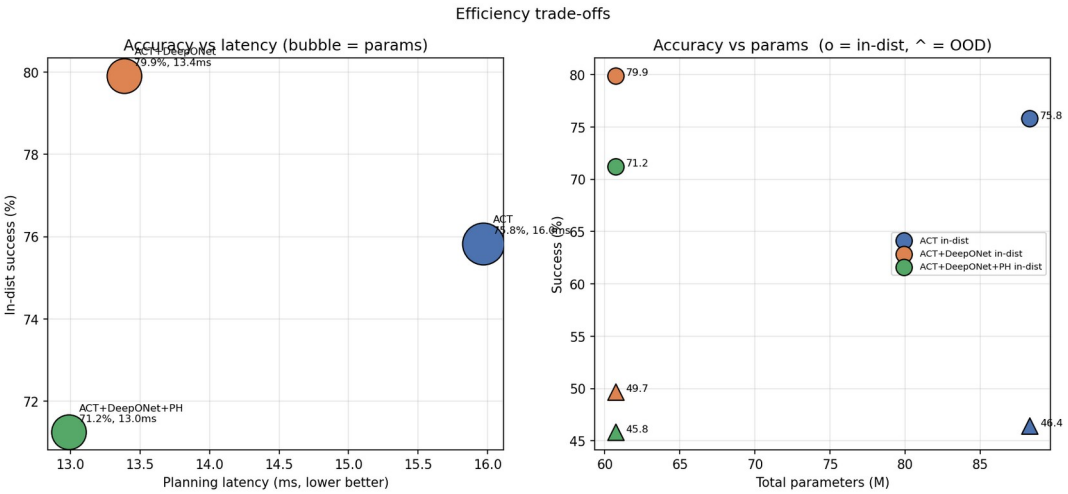
Suite x perturbation robustness heatmaps, one per variant (ACT).



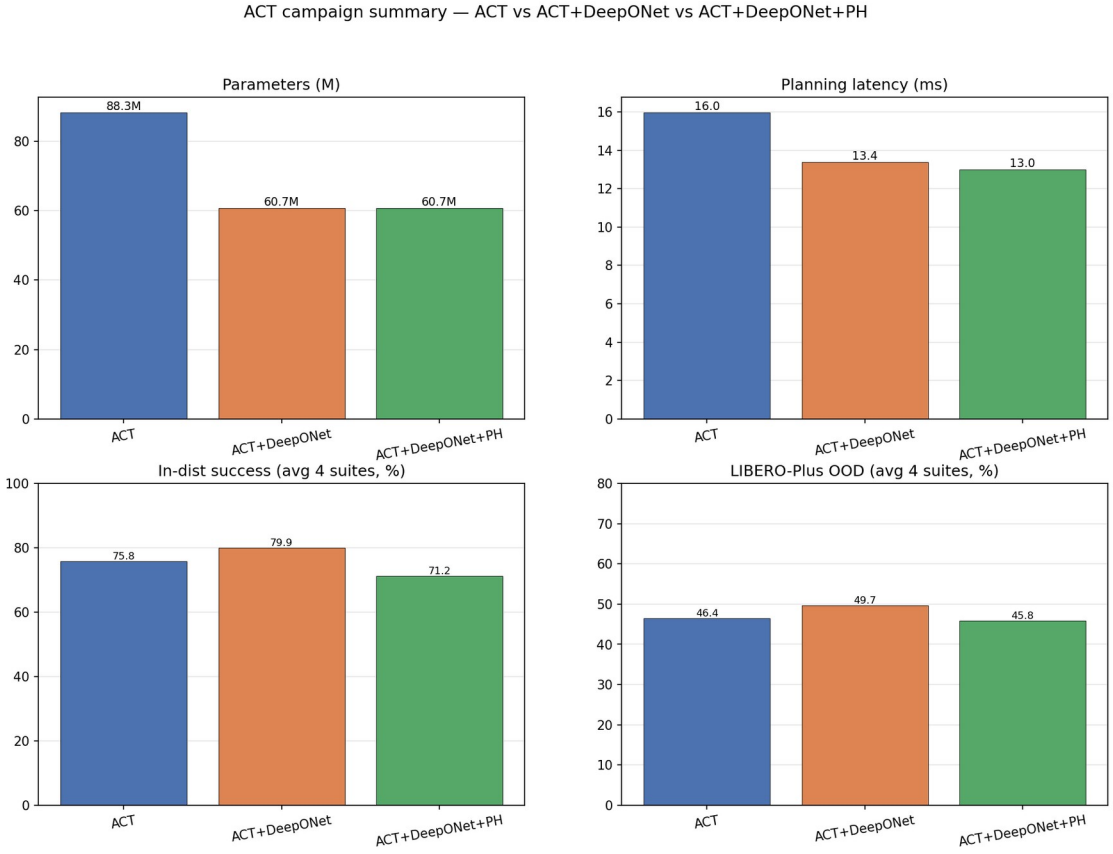
Radar view of the seven perturbation categories (ACT).

6.3.4 Efficiency

Variant	Total params	Action head	Planning latency	Per step (replan 5)	Control freq
ACT (baseline)	88.3M	37.8M	16.0 ms	3.90 ms	~257 Hz
ACT + DeepONet	60.7M	10.2M	13.4 ms	3.23 ms	~310 Hz
ACT + DeepONet + PH	60.7M	10.2M	13.0 ms	3.35 ms	~298 Hz



Efficiency: accuracy vs latency and vs parameter count (ACT).



Master summary: parameters, latency, in-distribution and robustness at a glance (ACT).

On ACT the operator head genuinely wins on every average axis: higher in-distribution, higher robustness, a 3.7× smaller head, and about 16% lower latency. This is the cleaner, more complete confirmation that we moved to ACT to get.

7. For completeness — pi0.5 and GR00T N1.6 (in progress)

To be thorough, we also ported the same operator head to two of the strongest available models. This is purely a state-of-the-art comparison, so we keep it short. **pi0.5** (~3.3B, PaliGemma backbone, native flow-matching head; Figure 9) and **GR00T N1.6** (~3B, Eagle backbone, native 32-layer diffusion head; Figure 10). In both, the backbone is frozen and we compare the native head against DeepONet and DeepONet + PH; in GR00T the head swap is controlled by an environment switch so the baseline is exactly the stock model. One practical note: GR00T's own evaluation uses **absolute** action commands while SmolVLA and pi0.5 use **relative** ones, so each must be evaluated in its own convention. These runs are still going, so the tables below are intentionally blank.

Figure 9. pi0.5 with the DeepONet action head (SOTA comparison)

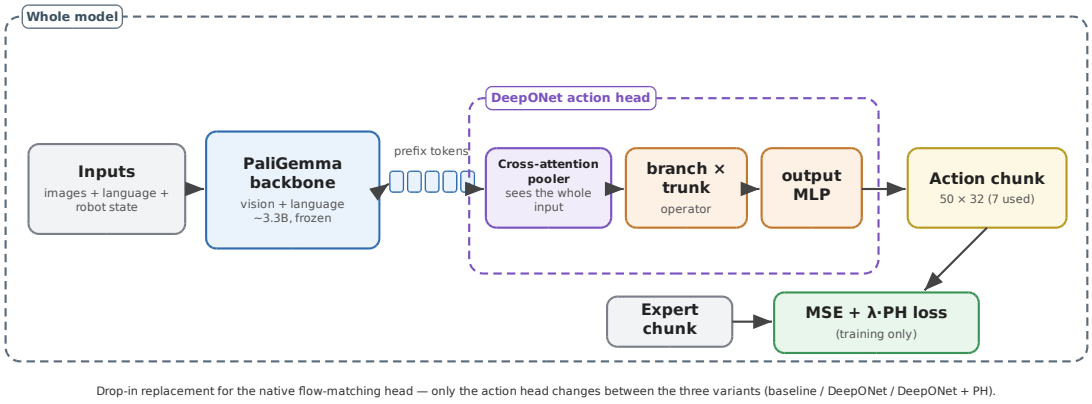


Figure 9. pi0.5 with the DeepONet action head. Inner boundary = the head we swap; outer boundary = the whole model; PH loss is training-only.

Figure 10. GR00T N1.6 with the DeepONet action head (SOTA comparison)

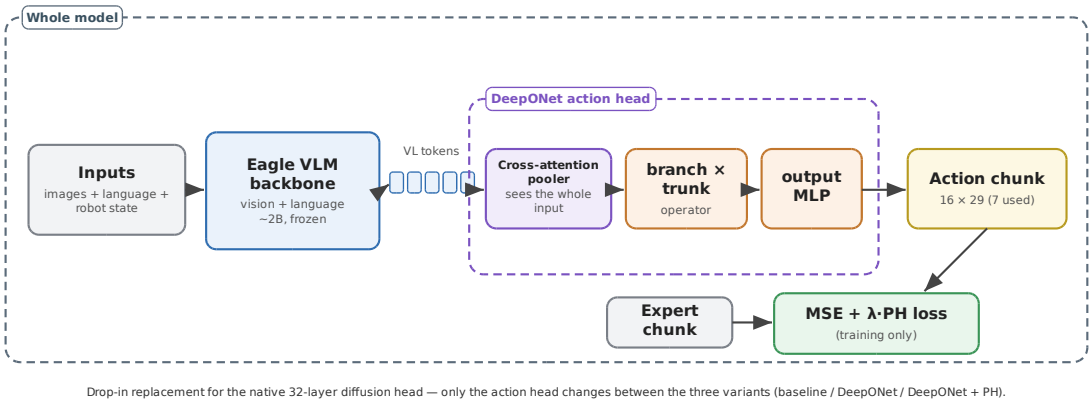


Figure 10. GR00T N1.6 with the DeepONet action head. Inner boundary = the head we swap; outer boundary = the whole model; PH loss is training-only.

pi0.5 (frozen backbone).

Head	Spatial	Object	Goal	Long	Average
Flow (baseline) — in-dist					
DeepONet — in-dist					
DeepONet + PH — in-dist					
Flow (baseline) — robustness					
DeepONet — robustness					
DeepONet + PH — robustness					

GR00T N1.6 (frozen backbone).

Head	Spatial	Object	Goal	Long	Average
Diffusion (baseline)					

— in-dist					
DeepONet — in-dist					
DeepONet + PH — in-dist					
Diffusion (baseline) — robustness					
DeepONet — robustness					
DeepONet + PH — robustness					

8. What we learned along the way (engineering notes)

The evaluation harness can quietly ruin your numbers. When we first ran robustness evaluation, one setting gave almost 0% on every perturbed task. It was not the model's fault: the perturbed-environment code sent our *relative* action commands into a controller expecting *absolute* targets, so the arm barely moved. Once we set the controller to relative mode and let the physics settle for a few frames after resetting, the numbers became sensible (this is why ACT-Object robustness went from ~0% to 53.6%).

Different models have different conventions. GR00T's own evaluation uses *absolute* actions, which is correct for it, so we deliberately did not carry the relative-action fix over to GR00T. Matching each model to its own convention is essential.

Two benchmark packages can clash. Normal LIBERO and LIBERO-Plus install under the same package name; we run the in-distribution and robustness evaluations as separate processes, each importing only its own version.

Be careful what a result really shows. Our own checks made us correct some early over-claims: the SmolVLA in-distribution difference is a tie, not a win; the operator's real, significant benefit is robustness; the PH loss significantly hurts robustness; and at a fixed budget the flow baseline slightly wins the four-suite in-distribution average. We report per-seed numbers, fix the training budget, and flag that the strong SmolVLA

robustness number is single-suite (which is part of why we did the full ACT study).

Disk space is a real constraint. The large checkpoints are several gigabytes each; a full disk once corrupted one mid-save. We now keep only the final checkpoint, delete each one after its evaluation is confirmed, and keep a safety margin.

9. Conclusion and future work

We brought two clean mathematical ideas — operator learning (DeepONet) and topology (persistent homology) — into modern robot vision-language-action models and tested them fairly, changing only the action head.

The operator head is the clear practical winner on efficiency: about 10M parameters and roughly 5× faster than the heavy flow head, with a single forward pass instead of a denoising loop. On the smaller ACT model it also wins on average in-distribution accuracy and robustness. On SmolVLA its honest story is more specific: in-distribution accuracy is a **statistical tie** with the flow baseline, but robustness to distribution shift is **more than doubled** and that difference is statistically significant. An ablation shows the robustness comes mainly from the cross-attention pooler (which lets the head see the input properly) and the operator structure, not from raw capacity — which is why our final module keeps the 3-block pooler and the operator merge, and drops the PH loss.

The persistent-homology loss, though mathematically appealing, did not help in either study — on SmolVLA it significantly reduced robustness — so we do not recommend it in this form. We think this is a genuinely useful negative result.

For future work we would like to finish the pi0.5 and GR00T runs and fill in the Section 7 tables; measure SmolVLA robustness on all four suites; try training the head with the backbone unfrozen; and explore better but still cheap topological losses, since our current PH term is only a fast surrogate.

References

1. T. Chen and H. Chen. *Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems.* IEEE Transactions on Neural Networks, 1995.
2. L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators.* Nature Machine Intelligence, 2021.

3. H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification.* Discrete & Computational Geometry, 2002.
4. G. Carlsson. *Topology and data.* Bulletin of the American Mathematical Society, 2009.
5. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. *Flow matching for generative modeling.* ICLR, 2023.
6. T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. *Learning fine-grained bimanual manipulation with low-cost hardware (ACT).* RSS, 2023.
7. Physical Intelligence. *pi0 / pi0.5: a vision-language-action flow model for general robot control.* 2024-2025.
8. NVIDIA. *GR00T N1 / N1.6: an open foundation model for humanoid and manipulation robots.* 2025.
9. B. Liu et al. *LIBERO: benchmarking knowledge transfer for lifelong robot learning.* NeurIPS Datasets and Benchmarks, 2023.
10. The LeRobot / SmolVLA project (Hugging Face), 2024-2025.

Appendix A. Action-head sizes

Model	Baseline head	DeepONet head	Reduction
SmolVLA	99.9M (flow)	10.4M	9.6×
ACT	37.8M (decoder)	10.2M	3.7×
pi0.5	flow expert	~11M	(large)
GR00T N1.6	~80-100M (diffusion)	~11M	~8-9×

Inside the DeepONet head, most parameters are in the cross-attention pooler (~7M) and the branch MLP (~3.3M); the trunk and output MLPs are tiny.

Appendix B. Variant names

- **M1 / baseline** — the model's original head (flow matching for SmolVLA and pi0.5; transformer decoder for ACT; diffusion for GR00T).
- **M3 / DeepONet** — our operator head replacing the baseline head.
- **M4 / DeepONet + PH** — the operator head with the persistent-homology loss added during training only.

Appendix C. Note on honesty and fairness

We gave our own method its strongest fair implementation but did not tune the evaluation to favour it. We report whatever the measurements show, including where our head does not win (the SmolVLA in-distribution tie, the fixed-budget four-suite result, and the fact that persistent homology hurts). This matters because the results are meant to stand up to careful statistical review.